

Git GitHub Essentials

Exercise Guide



Contents

1.	Course Introduction	1
2.	Introduction to Git	2
2.1.	Quick Lab – Git User Configuration	2
2.2.	Quick Lab – Git Help.....	3
2.3.	Quick Lab – Using Git Bash to create a first repository	4
3.	Branching and Merging	6
3.1.	Quick Lab – Create and Work on a Branch	6
3.2.	Quick Lab – Merge Branches.....	8
3.3.	Quick Lab – Deleting Branches.....	9
4.	Introduction to GitHub	10
4.1.	Quick Lab – Setup a GitHub Account.....	10
5.	Repository Creation and Setup	12
5.1.	Quick Lab – Create a GitHub Repository	12
5.2.	Quick Lab – Clone the Repository	13
6.	Linking to Visual Studio Code	14
6.1.	Quick Lab – Create and Manage Repos from VSC	14
6.2.	Quick Lab – Add extensions to VSC.....	18
7.	Pull Requests.....	23
7.1.	Quick Lab – Carry Out a Pull Request	23
8.	Managing Merge Conflicts.....	26
8.1.	Quick Lab – Managing Merge Conflicts	26
9.	GitHub Challenge.....	29
9.1.	Quick Lab – Managing Merge Conflicts	29

1. Course Introduction

There are no exercises for this section.

2. Introduction to Git

2.1. Quick Lab – Git User Configuration

Overview

Install Git Bash and use the command line to configure Git with your username and email.

- Check the settings to ensure these are set.

Steps

Getting Started with Git	
1	Download and install Git Bash using the following link. https://git-scm.com/downloads Click the Download button to get the appropriate version for your operating system (Windows, macOS, or Linux).
2	Run the Installer: • Open the downloaded file and follow the installation wizard. • During installation, ensure the option to install Git Bash is selected.
3	Verify the Installation: • Open Git Bash from your Start Menu (Windows) or Applications (macOS/Linux). • Run the following command to check the version: <code>git --version</code> • If the version displays, Git Bash is installed successfully.
Set Up Git with Your User Information	
4	configure your Git username and email by running the following 2 commands: <pre>git config --global user.name "Your Name" git config --global user.email "your.email@example.com"</pre>

2.2. Quick Lab – Git Help

Overview

Use your git bash command line to access git help to find out about some Git commands.

Steps

Using Git Help	
1	display the Git glossary by typing the following into Git Bash: <code>git help glossary</code>
2	List all Git commands: <code>git help -a</code>
3	Get information about Git configuration: <code>git help config</code>
4	Search git documentation for common concepts: <code>git help -g</code>

2.3. Quick Lab – Using Git Bash to create a first repository

Overview

Use your git bash command line to access git help to find out about some Git commands.

Steps

Create Your First Repository	
1	Open Git Bash and create and navigate to the folder where you want to create the repository using the cd command: <pre>cd path/to/your/folder</pre> For example: <pre>cd C:/Labs/GitTest</pre>
2	Initialize a new Git repository by running the following command: <pre>git init</pre> This will create a hidden .git folder that tracks your project.
3	Add files to your repository by either creating new ones or copying existing ones using any of the following options: • Use touch:<pre>touch myFile.txt</pre> • Use redirection to create a file (called myFile.txt) and add the text "Some Content" to it:<pre>echo "Some Content" > myFile.txt</pre> • Or use a text editor such as Notepad.exe:<pre>notepad myFile.txt</pre> • To copy a file from one location to another, use the git cp command:<pre>cp source_filepath destination_filepath</pre>
4	Add all the files to the repository's staging area: <pre>git add -A</pre>
5	Commit the changes: <pre>git commit -m "Initial commit"</pre>

6	Verify the repository's status: <code>git status</code>
7	View the commit history: <code>git log</code>

3. Branching and Merging

3.1. Quick Lab – Create and Work on a Branch

Overview

Create a new branch, add a new file to it, commit the changes, return to the master, ensure new file is **not** in the folder and check the repository's history.

Steps

Create a new Branch	
1	Return to the repository you worked on in the previous lab
2	Create and checkout a new branch called new-feature: <pre>git branch new-feature git checkout new-feature</pre> Or combine the two commands into one using the -b option: <pre>git checkout -b new-feature</pre>
3	Make some changes to the new branch by adding a new file: GITBASH: <pre>touch myNewFeaturesFile.txt echo "Some New Features Content" > myNewFeaturesFile.txt</pre> WINDOWS: Create a new file using File Explorer and add some text to it Feel free to add and edit some additional files if you want.
4	Commit the changes: <pre>git add . git commit -m "Added a file on new branch"</pre>
5	Check the content of the repository's folder and ensure the new features file is present. <pre>ls</pre> OR

	<code>dir</code>
Checkout the main branch	
6	Return to the main branch: <code>git checkout master</code> Note: the main branch may be called <code>main</code>
7	Check the content of the repository's folder and ensure the new features file is not present. <code>ls</code>
8	Check the log history: <code>git log</code>

3.2. Quick Lab – Merge Branches

Overview

Merge the new-feature branch back into main.

Steps

Switch to the target branch	
1	Return to the repository you worked on in the previous lab
2	Switch to the branch you want to merge into (e.g., master or possibly main). <pre>git checkout master</pre> Note: the main branch may be called main
Merge the Source Branch into the Target Branch	
3	Use the git merge command to merge changes from the branch you were working on (new-feature) into the current branch. <pre>git merge new-feature</pre> This will merge the changes from new-feature into main. If there are no conflicts, Git will perform the merge automatically. If there were to be any conflicts, Git would prompt you to resolve them manually before completing the merge (see more about how to do this in session 08).
4	Check the contents of the folder and note it contains the myNewFeaturesFile.txt file: <pre>ls</pre>

3.3. Quick Lab – Deleting Branches

Overview

Delete the new-feature branch.

Steps

Deleting a Branch	
1	Use the branch command to see all current branches: <code>git branch</code>
2	Make sure the current branch is <code>master</code> (or possibly <code>main</code>): <code>git checkout master</code> Note: the main branch may be called <code>main</code>
2	Once you've successfully merged a branch, you can delete it if you no longer need it. <code>git branch -d new-feature</code>
3	Try to delete the main branch (the attempt should fail): <code>git branch -d master</code> Note: the main branch may be called <code>main</code>
4	Check the log history: <code>git log</code>

4. Introduction to GitHub

4.1. Quick Lab – Setup a GitHub Account

Overview

Follow these simple steps to create and configure your GitHub account.

Steps

Sign Up to GitHub	
1	Open a web browser and browse to https://github.com
2	Click the "Sign up" button in the top-right corner of the GitHub homepage.
3	Fill in the required information: • Username: Choose a unique username that represents you. • Email Address: Provide a valid email address (used for account recovery and notifications). It's best to use the same address as the one you used to configure Git. However, GitHub allows you to add as many email addresses to your account as you like. If your Git email address is different, you will need to add it to your GitHub account settings in order to connect your commits to your account. For more information see Adding an email address to your GitHub account - GitHub Docs. • Password: Create a strong password.
4	Click Create Account
5	Verify your email address by checking your email inbox for a verification email from GitHub. When it arrives, open the email and click the verification link to confirm your email address.
6	Choose a plan (GitHub offers both free and paid plans). The free plan is more than adequate for you to fully get to grips with GitHub and complete the course.

7	Optionally complete the onboarding process by specifying why you are using GitHub (personal, professional, educational...). There is no need to enable any of the recommended features.
---	---

5. Repository Creation and Setup

5.1. Quick Lab – Create a GitHub Repository

Overview

Follow these simple steps to create and configure a new GitHub repository.

Steps

To Create a GitHub Repository	
1	Open a web browser and browse to https://github.com and sign in
2	Navigate to the "New Repository" page by clicking the "+" icon in the top-right menu and select "New repository". Or Click on your profile picture in the top-right corner of the page. From the dropdown menu, select "Your repositories". Click the green "New" button on the right side of the repositories page.
3	Complete the repository details by specifying: Repository Name: myFirstGitHubRepository Description (Optional): Leave this empty Visibility: Public Initialize with a README: Check this box Add .gitignore: Select a language template (if in doubt select Python)
4	Create the repository by clicking the "Create Repository" button. GitHub will create the repository and redirect you to its main page.

5.2. Quick Lab – Clone the Repository

Overview

Follow these simple steps to clone the GitHub repository.

Steps

Configure Git to use the GitHub repository and clone its content	
5	In GitHub click on the green Code dropdown button on the repository page and copy the HTTPS link.
6	Create a new Git repository on your local machine by running Git Bash and changing to a relevant folder using the <code>cd</code> (change directory) and <code>mkdir</code> (make directory) functions as appropriate. Then run: <code>git init</code>
7	Clone the repository to your local machine by entering: <code>git clone <repository-url></code> Where <code><repository-url></code> is the URL you copied in step 5.
8	Ensure the README and .gitignore files have been downloaded to the folder. <code>ls -a</code> The "a" switch shows all files and folders.

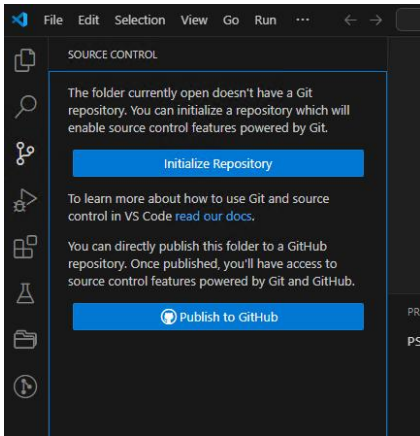
6. Linking to Visual Studio Code

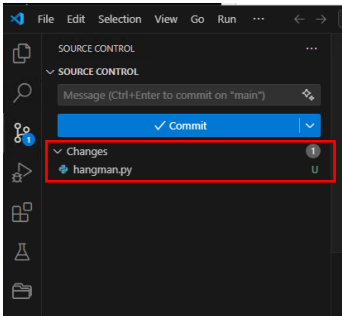
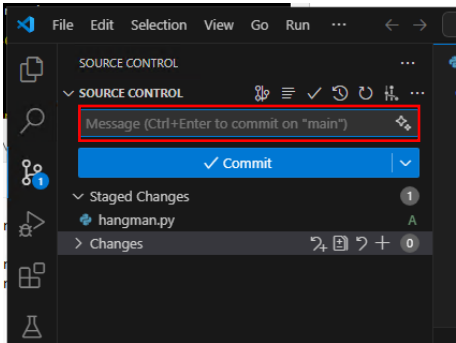
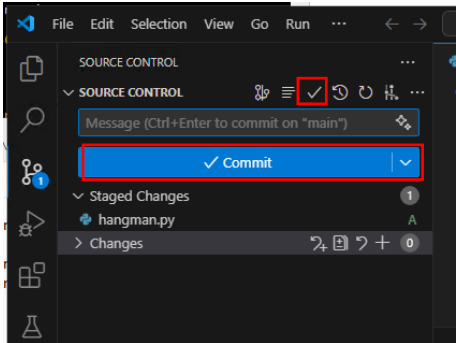
6.1. Quick Lab – Create and Manage Repos from VSC

Overview

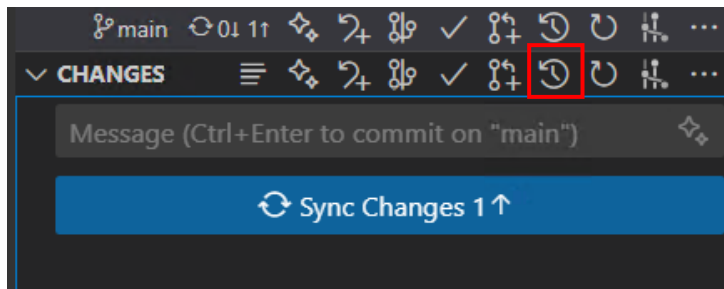
Create a new Git repository from a local Python project using Visual Studio Code (VS Code) and push it to GitHub as a new repository.

Steps

Initialize a Git Repository in Your Local Python Project	
1	Copy the Hangman.py file located in C:/Labs/Games to a folder of your choice.
2	Launch VS Code and open the folder containing your Python code.:
3	Open the Source Control panel in the Activity Bar on the left-hand side of the screen (it looks like a branch with nodes).
4	If your folder is not yet a Git repository, you can initialize it by clicking "Initialize Repository" so it runs <code>git init</code> in your project folder.  A screenshot of the Visual Studio Code Source Control panel. The panel has a dark theme. At the top, it says 'SOURCE CONTROL'. Below that, a message states: 'The folder currently open doesn't have a Git repository. You can initialize a repository which will enable source control features powered by Git.' There is a blue button labeled 'Initialize Repository'. Below this, it says 'To learn more about how to use Git and source control in VS Code read our docs.' and 'You can directly publish this folder to a GitHub repository. Once published, you'll have access to source control features powered by Git and GitHub.' There is another blue button labeled 'Publish to GitHub' with a GitHub logo icon.
5	View changes by looking at the Source Control panel, where you'll see a list of files with changes.

	
6	Stage the changes by hovering over the file you want to stage and clicking the + icon that appears next to the file name. Or, right-click the file and select "Stage Changes". To stage all changes at once, click the + icon next to the Changes heading.
7	Commit the changes by firstly writing "First commit" in the Message text box located above the Commit button.  And then, press the Commit button or click the checkmark icon (a tick symbol) at the top of the Source Control panel. 
8	Verify the commit by clicking icon with an analogue clock with an anticlockwise pointing arrow (or if not immediately available try the period ellipses (...)) in the

Source Control panel and selecting "Git: View History (git log)" (Note, this requires an extension like GitLens).



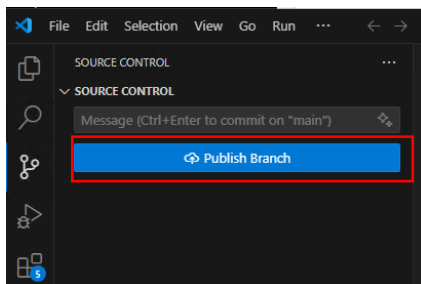
Alternatively, use VSCode's terminal window (Shown via the View menu's Terminal option) and running:

```
git log
```

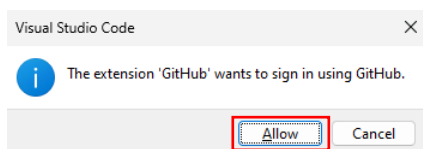
Push the repository to GitHub

9 Make sure there is an appropriate GitHub repository for the code to be pushed to (you could use the one created in Quick Lab 5.1 or create a new one).

10 Click the Publish Branch button located in the Source Control panel.



Click the "Allow" button on the message box if it appears:



If prompted select a user to authorise VSCode's request and grant any additional permissions.

11 Select the option that will publish to a public GitHub repository. This option should create a new repository on your GitHub account.

12 Navigate to your GitHub home page. Click on your profile picture in the top-right corner of the page and select the "Repositories" option. You should see your newly

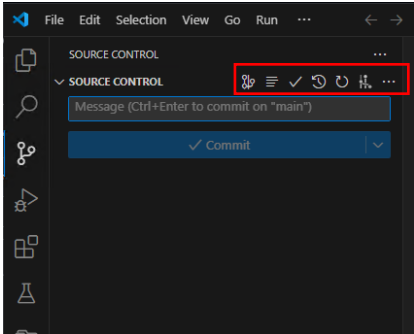
	created repository on the top of the list. Click on the repository name and make sure the hangman.py file has been uploaded.
--	--

6.2. Quick Lab – Add extensions to VSC

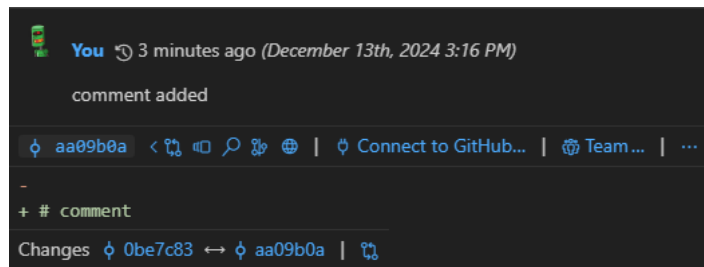
Overview

Add some of the recommended Git extensions to VSCode to enhance the ways you can interact with repositories.

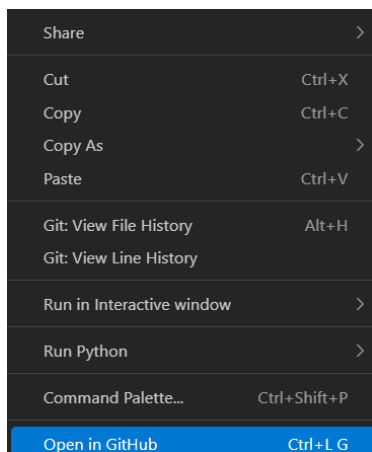
Steps

Add Some Extensions to VSCode	
1	Open the Extensions panel in the Activity Bar on the left-hand side of the screen (it looks like a branch with stacked boxes).
2	Type Git Blame into the search box and click on its install button
3	Repeat step 2 for Git Extension Pack (which will install "Open in GitHub, Bitbucket and GitLab", "GitLens" and "Git History")
4	Repeat step 2 for Git Graph.
5	Return to viewing VSCode's Source Control panel. Notice the additional options that have appeared on its top menu.  Hover your mouse over the various options to see some text that describes what they do.
6	Add a line of comment to the hangman.py file: <pre># comment</pre> Stage and commit the change (don't forget the commit message)
7	Hover the cursor over the comment and click the mouse. Notice text appears (to the right) that tells you who authored the line and when the change was made. Hovering

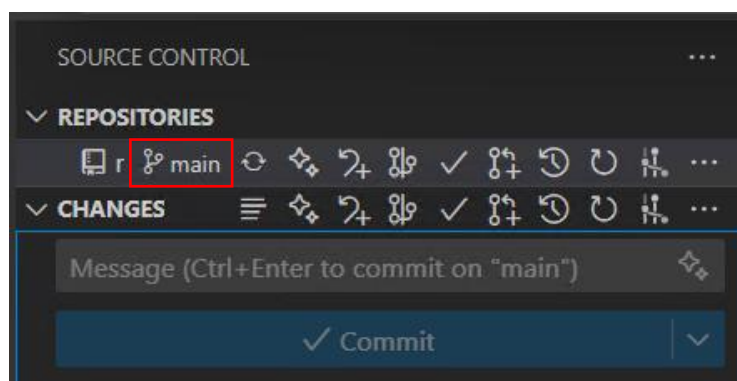
the mouse over this text causes a pop-up window to appear showing further details and links.



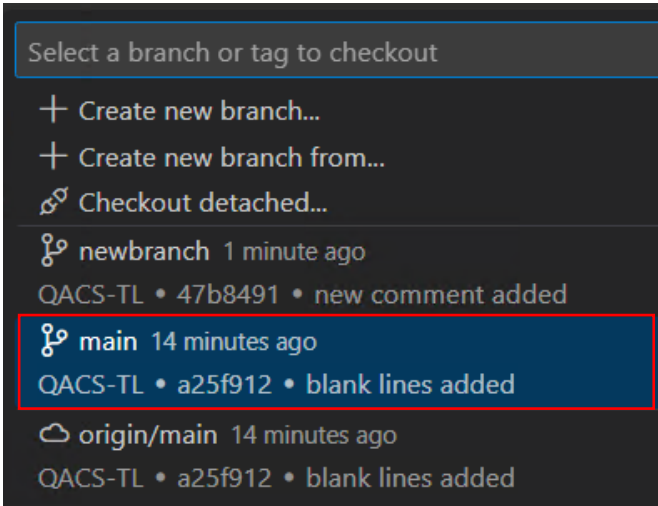
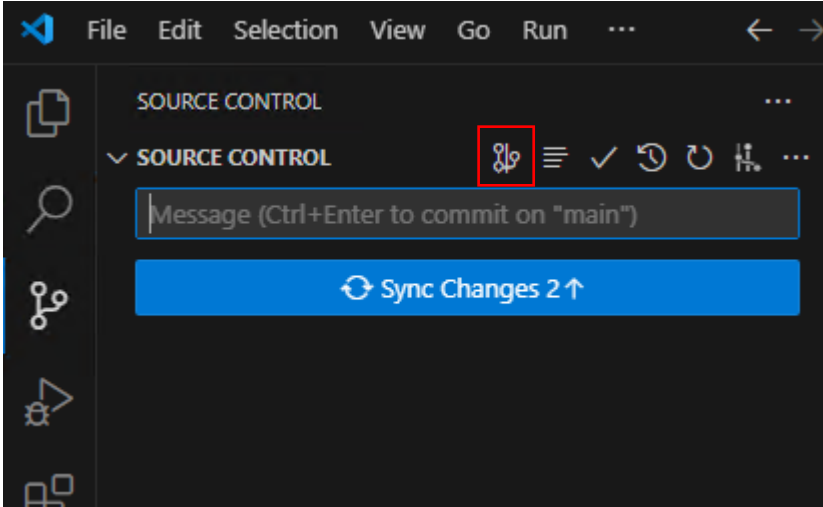
- 8 Right click on a piece of code and select "Open in GitHub. Notice the code for the specified page opens up in GitHub.

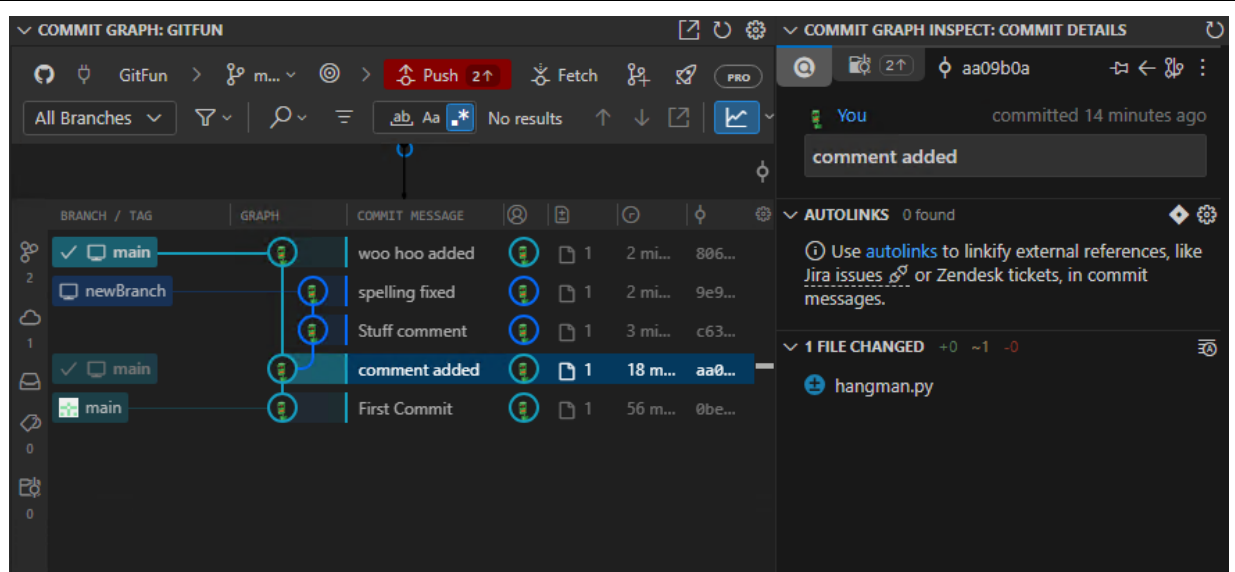


- 9 In the Repositories section of the Source Control window select the "main, Checkout Branch/Tag" button and create a new branch called "newbranch".

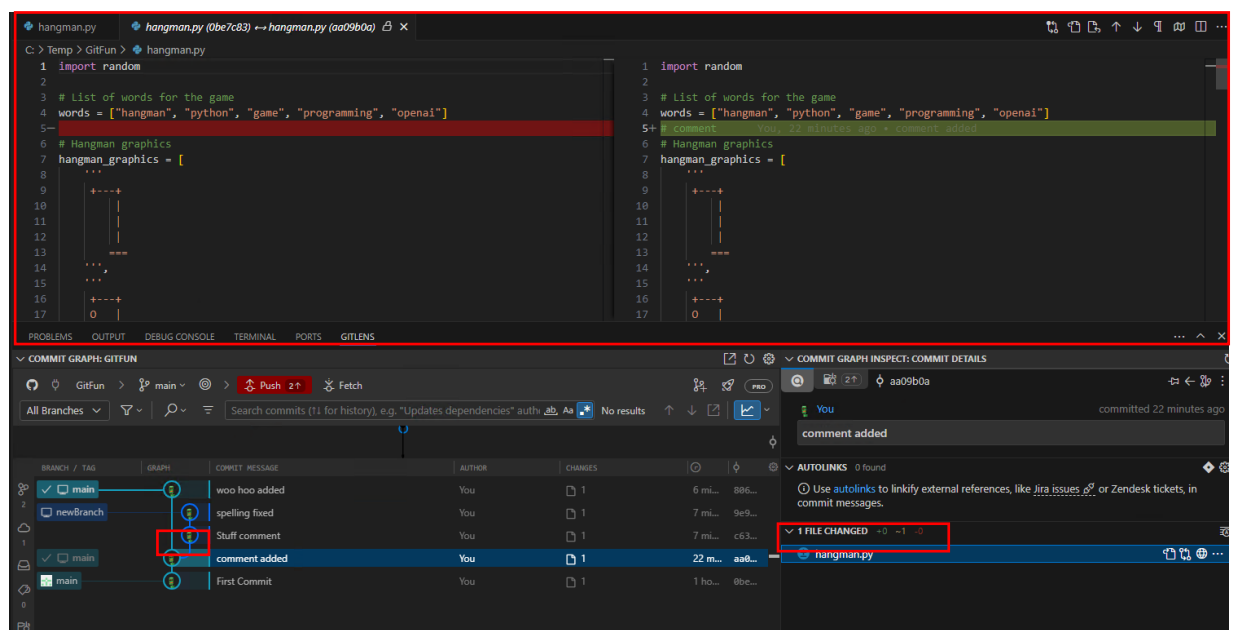


Edit the hangman.py code a few times, staging and committing changes as you go. Return to the main branch (clicking the same button as before but select the main branch)

	 Make some further edits and commits to this (main) branch but, for the time being, be gentle and don't edit or delete a line that you amended when operating on the "newbranch" branch..
10	Click on the Show Commit Graph button on the Source Control Panel's menu. 
11	Look at the Commit Graph that gets displayed (something like the following):



Notice it visually shows the branches and commits with details of the author, dates and times the changes were made. Clicking on a commit reveals information of the change that was made in the right-hand pane. Clicking on this opens up a before and after pair of windows that provide a before and after snapshot of what happened:



- | | |
|----|--|
| 12 | Click on the other buttons on the Source Control panels' menu and see if you can work out what they are showing and doing. |
| 13 | DO NOT push the changes to GitHub. Wait until the next Quick Lab. |

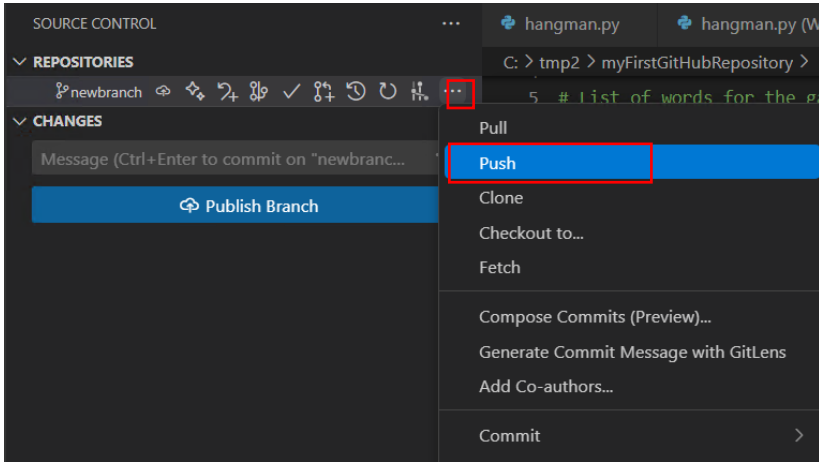
7. Pull Requests

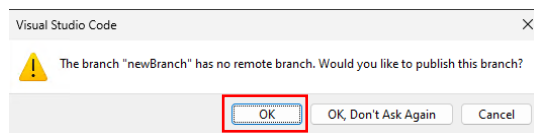
7.1. Quick Lab – Carry Out a Pull Request

Overview

Push a changed and committed branch to GitHub and carry out a "clean" pull request with no merge issues.

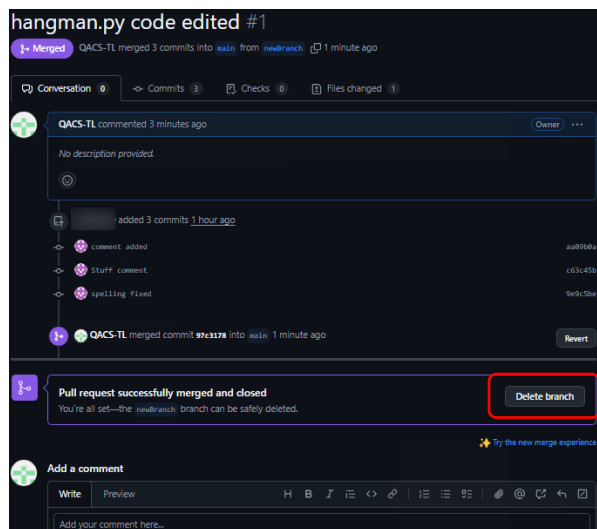
Steps

Push the latest version of the Committed changes made to the Hangman.py File	
1	Return the VSCode and the project that contains the Hangman.py file. Make sure the current branch is "newbranch" (the new one you created in step 9 of the previous quick lab).
2	Make sure there are a number of committed changes (on the "newbranch" branch) which won't cause any merge issues and ensure the changes are ready to be pushed to GitHub. Note, there should be no new commits on the local main branch at this point)
3	Push newbranch to the GitHub repository by clicking the period ellipses (...) in the Repositories (or Changes) panel and selecting Push:  The screenshot shows the VS Code Source Control panel. The 'REPOSITORIES' section is expanded, showing a branch named 'newbranch'. A context menu is open for 'newbranch', and the 'Push' option is highlighted in blue. The 'CHANGES' section below shows a commit message input field and a 'Publish Branch' button.
4	Answer OK to the pop-up message that says 'The branch "newBranch" has no remote branch. Would you like to publish this branch?'



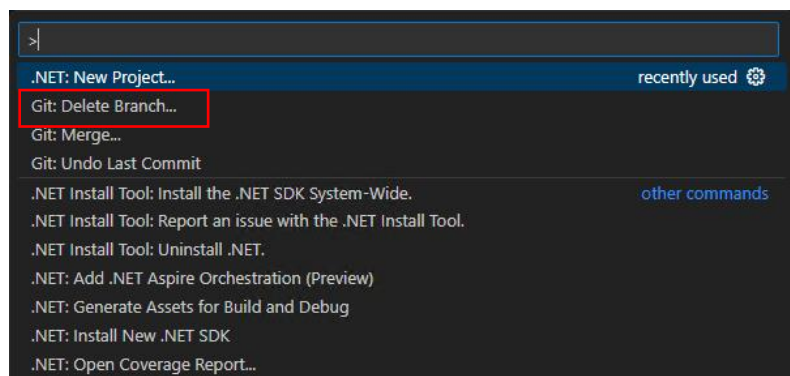
Create a Pull Request on GitHub

5	Navigate to the GitHub repository page in your browser.
6	Open the Pull Request Page (GitHub will usually show a banner suggesting you create a pull request after pushing a new branch). Click "Compare & pull request". If no banner appears, go to the Pull Requests tab and click "New Pull Request".
7	Fill in the Pull Request Details: Ensure the base branch (main) and compare branch (newbranch) are correct and "able to merge". Add a title and description explaining the changes (something like "hangman.py code edited")
8	Click the "Create pull request" button.
9	Review and Merge the Pull Request. In the real world, Team members or collaborators can review the code and suggest changes. If changes are requested, implement them locally, commit them, and push them to the same branch. The Pull Request (PR) will automatically update with the new commits.
10	Merge the Pull Request by clicking the "Merge pull request" button on the GitHub Pull Request page. Confirm the merge by clicking "Confirm merge".
11	After merging, GitHub will offer an option to delete the branch. Click "Delete branch".



If not prompted, you can manually delete it under the "Branches" tab.

- 12 Return to VSCode and make sure you are working on the main branch (click the "Main, Checkout Branch/Tag" button and select the main branch). You can now delete "newbranch" locally by pressing the "F1" key and selecting the "Git: Delete Branch..." option from the drop down.



And then selecting newbranch.

- 13 Run `git log` to confirm all is well.

8. Managing Merge Conflicts

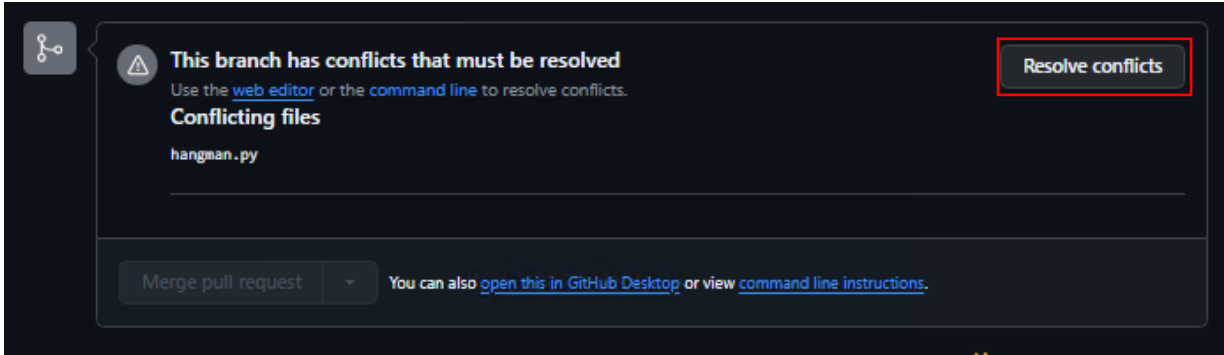
8.1. Quick Lab – Managing Merge Conflicts

Overview

Handling a Pull Request (PR) with conflicts involves identifying the issues, resolving them, and ensuring the codebase is stable.

Steps

Make changes to and push the latest version of the committed versions of the Hangman.py file to GitHub	
1	In the VSCode project create a new branch (called "newbranch") and edit a line in the Hangman.py file and remember the line number. Save, stage and commit the changes.
2	Make sure you are on the main branch in VSCode. Make some changes to the same line of code in Hangman.py file. Save, stage and commit the changes.
3	Switch to the "newbranch" branch and push the changes to GitHub. Answer OK to the pop-up message that says 'The branch "newBranch" has no remote branch. Would you like to publish this branch?'
4	Switch back to the "main" branch and push the changes to GitHub.
5	Navigate to the GitHub repository page in your browser.
6	Open the Pull Request Page by clicking the "Compare & pull request" button. If no banner appears, go to the Pull Requests tab and click "New Pull Request".
7	Ensure the base branch (Something main) and compare branch (new branch) are correct. You will probably get a warning that the branches can't be automatically merged but note that you can still create a Pull Request by clicking the "Create Pull request button". Do this now.
8	Give the request a Title and press the "Create Pull Request" button

9	GitHub will display a message like <i>"This branch has conflicts that must be resolved"</i>. Click Resolve Conflicts to view the files with conflicts. 
10	Understand the Conflict: Conflicts occur when changes from the feature branch and the main branch overlap. Git marks the conflicts in the file with: <pre> <<<<<<< newbranch (Code from the newbranch branch) ===== (Code from the main branch) >>>>>>> main </pre> You'll notice the "Mark as resolved" button is disabled.
11	Note: We are going to resolve the issue directly in the current GitHub window, but it is often better for control to handle them locally. You could do this by running <code>git fetch origin</code> and <code>git checkout newbranch</code> commands in VSCode's terminal window. And then running <code>git merge newbranch</code> from the main branch and use the IDE's tools to resolve the conflicts (in VSC this is the Resolve Tools Merge Editor).
12	Edit the code in the GitHub window by removing all the conflict markers (<code><<<<<<<</code>, <code>=====</code>, <code>>>>>>>></code> and branch names) and changing what is left to reflect the "correct" version. You'll notice the "Mark as resolved" button has now been enabled.
13	When you are happy with your changes click the "Mark as resolved" button. And then the "Commit Merge" button
14	You should see a message that says, "This branch has no conflicts with the base branch" and a green Merge pull request button which you should press. You can finally click the "Confirm Merge" button

15	All should be well, and you are invited to press the Delete branch button which will be a safe thing for you to do.
16	Delete the branch locally (in VSC) by firstly making sure you are working on the "main" branch, pressing the "F1" key and then selecting the "Git: Delete Branch..." option from the drop down.
17	Run <code>git log</code> to confirm all is well.

9. GitHub Challenge

9.1. Quick Lab – Managing Merge Conflicts

Challenge

Your tutor will have paired you up and sent you to a breakout room. You are tasked to collaboratively create some code (using a programming language of your choice). Each one of you needs to create a unique code file and you should also share a file. You don't need to write any valid code (lines of comment, plain English or gibberish will be fine) but both of you should edit your unique file and the shared file. The repository should also include a README file which both of you should edit and a .gitignore file.

You should start by creating a single GitHub repository. Obviously, only one of you can do this. The creator will also need to add the other one of the pair as a collaborator (which means the sharing of your GitHub user-name or email address). You can do this via the Settings cog menu option. Once done, you should both clone the repository down to your local machines and create your own branches which you should work with to add your unique files and edit the shared ones.

The principal aim is to ensure the main branch on GitHub is **never** compromised. If a Pull request identifies a conflict, then it is generally better to fetch and review the issues locally and having resolved the conflicts to commit and push them back to GitHub.



QA.com